

STATSIG lecture 12

Adaptive filters 2

RLS algorithm

Disclaimer:

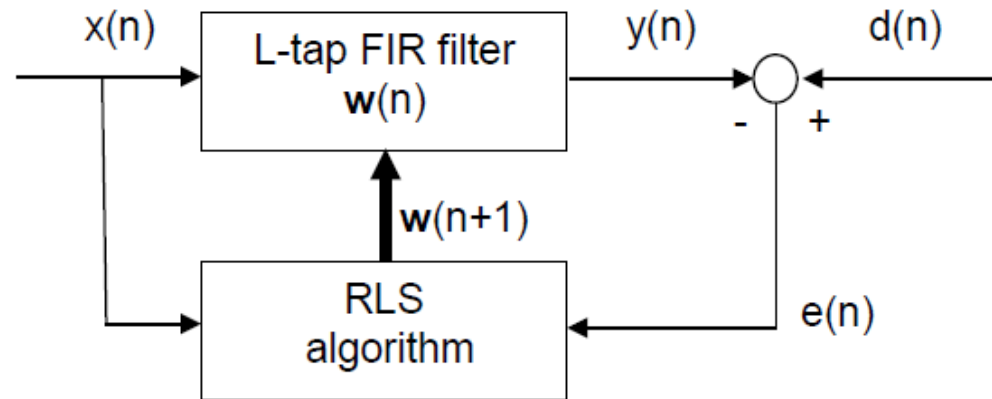
**Please note that these slides are only a brief overview of the topic.
Many more details will be explained on the blackboard and with
MATLAB examples**

Adaptive filters and RLS algorithm (faster convergence than LMS, higher complexity)

Applicable for non-stationary (not LTI) systems and signals.

Is specified in terms of cost function (MSE) in time-domain and not like standard digital filters in terms of cutoff frequencies.

Generic model reviewed
(detailed definitions
on blackboard)



input signal $x(n)$

standard FIR-filter with L coefficients in vector $w(n)$

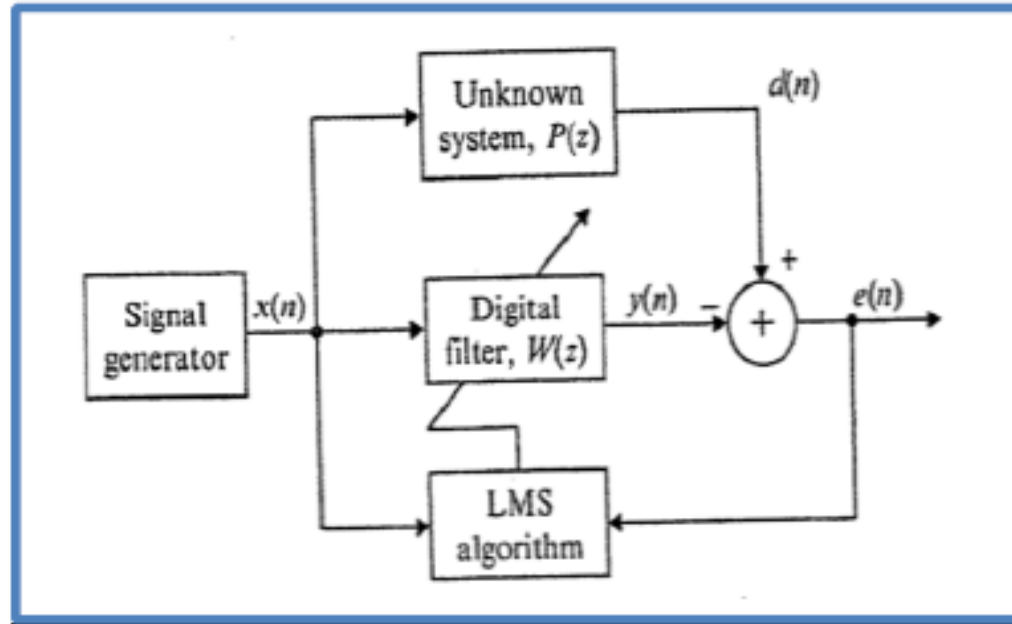
output from filter $y(n)$

"desired signal" $d(n)$ (generated in different ways dependent on actual application)

error signal $e(n) = d(n) - y(n)$

adaptive RLS algorithm calculates filter coefficients $w(n+1)$ for next sample period

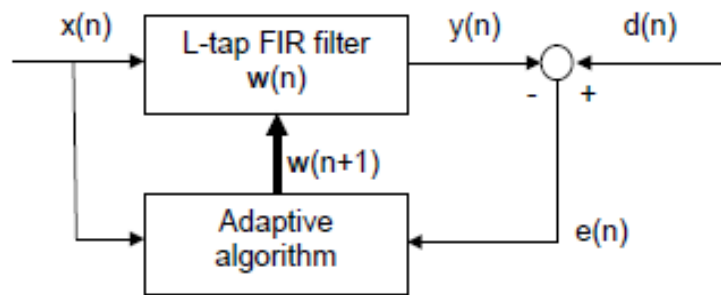
Application: Adaptive system identification of time-varying system
Other applications follows later in lecture



Domain:
Acoustics, Vibration, Communication, Control, ...

Operation:
Adaptive algorithm minimizes $e(n) \Rightarrow W(z) \cong P(z)$

Optimal adaptive FIR filter – Wiener Hopf reviewed



Cost-function

$$\begin{aligned}\xi(n) &= E[e^2(n)] = E\left[\left(d(n) - \mathbf{w}^T(n)\mathbf{x}(n)\right)^2\right] \\ &= E[d^2(n)] - 2\mathbf{w}^T(n)E[d(n)\mathbf{x}(n)] + \mathbf{w}^T(n)E[\mathbf{x}(n)\mathbf{x}^T(n)]\mathbf{w}(n)\end{aligned}$$

Autocorrelation matrix

$$\mathbf{R}_{xx}(n) = E[\mathbf{x}(n)\mathbf{x}^T(n)] = \begin{bmatrix} r_{xx}(0) & \cdots & r_{xx}(L-1) \\ \vdots & \ddots & \vdots \\ r_{xx}(L-1) & \cdots & r_{xx}(0) \end{bmatrix}$$

Cross correlation vector

$$\mathbf{r}_{dx}(n) = E[d(n)\mathbf{x}(n)] = \begin{bmatrix} r_{dx}(0) \\ \vdots \\ r_{dx}(L-1) \end{bmatrix}$$

Giving

$$\xi(n) = E[d^2(n)] - 2\mathbf{w}^T(n)\mathbf{r}_{dx}(n) + \mathbf{w}^T(n)\mathbf{R}_{xx}(n)\mathbf{w}(n)$$

Optimization

$$\nabla \xi(n) = \frac{\partial \xi}{\partial \mathbf{w}} = 0 - 2\mathbf{r}_{dx}(n) + 2\mathbf{R}_{xx}(n)\mathbf{w}(n) = \mathbf{0}$$

Optimal filter (Wiener Hopf solution)

$$\mathbf{w}(n) = \mathbf{w}_{opt}(n) = \mathbf{R}_{xx}^{-1}(n) \mathbf{r}_{dx}(n)$$

LMS algorithm **does not use** the Wiener-Hopf solution for filter coefficients

Instead an iterative search towards optimal filter coefficients is performed

RLS, however, **does use** the Wiener-Hopf solution, but instead of a direct calculation of optimal filter coefficients with high computational complexity (must be performed each sample period), the RLS calculates the required inverse of the autocorrelation matrix in a recursive way

Steps towards RLS (Recursive Least Squares) algorithm for optimal filter coefficients (details on blackboard)

1. Starting point (Wiener-Hopf equation) $w(n) = R_{xx}^{-1}(n)r_{dx}(n)$
2. From ensemble averages to time averages (estimates) $R_{xx}(n) \rightarrow \hat{R}_{xx}(n), \quad r_{dx}(n) \rightarrow \hat{r}_{dx}(n)$
3. Limitation of estimates "infinite memory" by introducing "forgetting factor", $\alpha, \quad 0 < \alpha < 1$

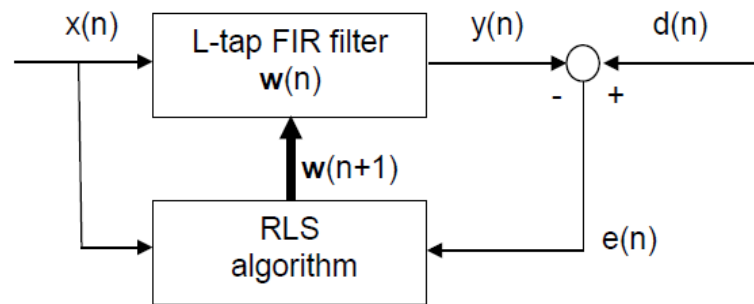
Modified cost function $\xi(n) = \sum_{k=0}^n e^2(k) \rightarrow \xi(n) = \sum_{k=0}^n \alpha^{n-k} e^2(k)$

4. Recursive form of estimates $\hat{R}_{xx}(n) = f\{\hat{R}_{xx}(n-1), x(n)\}, \quad \hat{r}_{dx}(n) = f\{\hat{r}_{dx}(n-1), x(n), d(n)\}$
5. Recursive equation for filter coefficients $w(n) = w(n-1) + \hat{R}_{xx}^{-1}(n)x(n)e(n)$
6. Recursive form for autocorrelation matrix inverse

Using Shermann-Morrison inversion equation $(A + uv^T)^{-1} = A^{-1} - \frac{A^{-1}uv^T A^{-1}}{1 + v^T A^{-1}u}$

is here found (after simplification) $\hat{R}_{xx}^{-1}(n) = \frac{1}{\alpha} \left[\hat{R}_{xx}^{-1}(n-1) - \frac{\hat{R}_{xx}^{-1}(n-1)x(n)x^T(n)\hat{R}_{xx}^{-1}(n-1)}{\alpha + x^T(n)\hat{R}_{xx}^{-1}(n-1)x(n)} \right]$

RLS organized as an algorithm



Parameters

L = filter length

α = forgetting factor, $0 < \alpha < 1$

for "MSE" $\xi(n) = \sum_{k=0}^n \alpha^{n-k} e^2(k)$

Init

$$\mathbf{w}(0) = \begin{bmatrix} w_0(0) \\ \vdots \\ w_{L-1}(0) \end{bmatrix} = \mathbf{0}, \quad \mathbf{x}(0) = \begin{bmatrix} x(0) \\ \vdots \\ x(-(L-1)) \end{bmatrix} = \mathbf{0}$$

$$\mathbf{R}_{xx}(0) = \delta \mathbf{I}_L \quad \text{where } 0 < \delta < 1 \Rightarrow$$

$$\mathbf{R}_{xx}^{-1}(0) = \frac{1}{\delta} \mathbf{I}_L$$

$$n = 1$$

Loop

Get data

$$\mathbf{x}(n) = \begin{bmatrix} x(n) \\ \vdots \\ x(n-(L-1)) \end{bmatrix}$$

$$d(n)$$

Predict

$$\hat{d}(n) = y(n) = \mathbf{w}^T(n-1) \mathbf{x}(n)$$

Error

$$e(n) = d(n) - \hat{d}(n)$$

Recursion

$$\mathbf{R}_{xx}^{-1}(n) = \frac{1}{\alpha} \left[\mathbf{R}_{xx}^{-1}(n-1) - \frac{\mathbf{R}_{xx}^{-1}(n-1) \mathbf{x}(n) \mathbf{x}^T(n) \mathbf{R}_{xx}^{-1}(n-1)}{\alpha + \mathbf{x}^T(n) \mathbf{R}_{xx}^{-1}(n-1) \mathbf{x}(n)} \right]$$

Update filter

$$\mathbf{w}(n) = \mathbf{w}(n-1) + \mathbf{R}_{xx}^{-1}(n) \mathbf{x}(n) e(n)$$

Iterate

$$n = n + 1$$

End

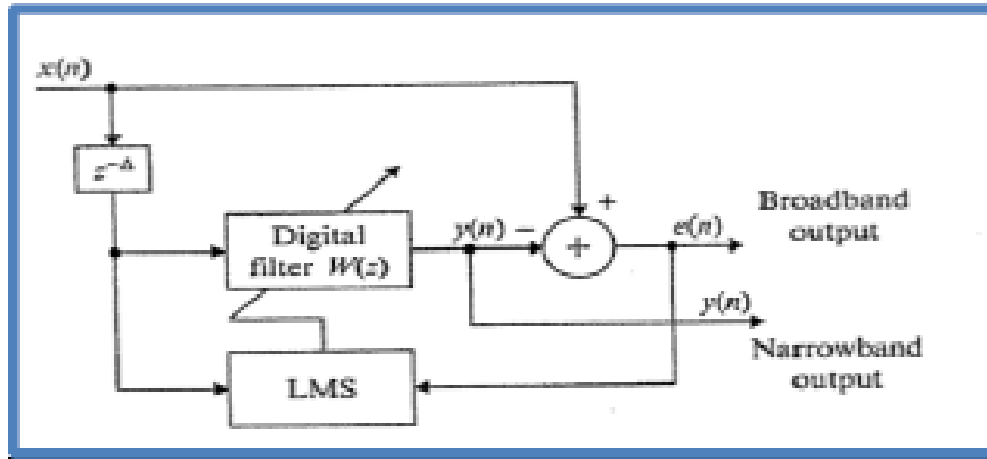
Computational complexity $\mathcal{O}(L^2)$

RLS algorithm performance measures

(blackboard, pdf-file "RLS algorithm and performance", MATLAB demos)

Application: Adaptive Linear Prediction

Separation of narrowband signal parts (classically the actual signal) and wideband signal parts (classically wideband ("white") noise)



Domain:

Communications, ...

Operation:

Signal $x(n) = s(n) + v(n)$ consists of narrowband part, $s(n)$, and wideband part, $v(n)$

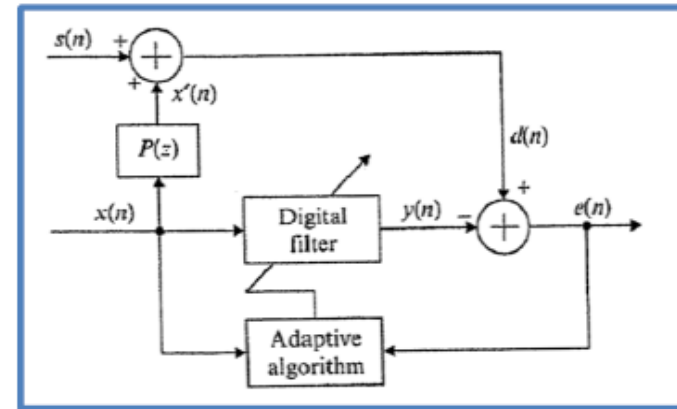
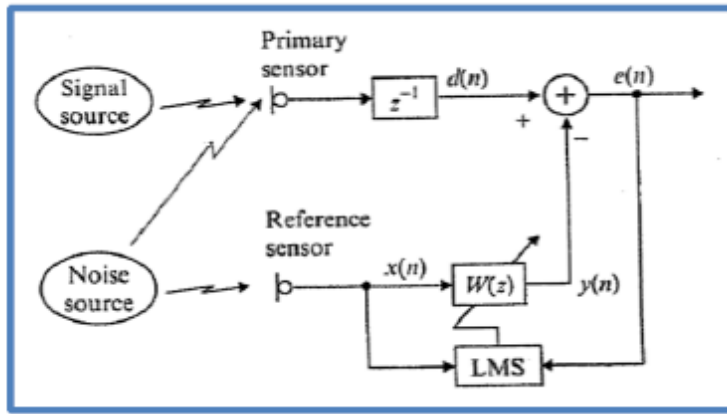
Adaptive algorithm minimizes $e(n)$ by linear prediction of $x(n)$ from $x(n - \Delta)$

The delay, Δ , is chosen big enough to ensure $R_{vv}(k) \cong 0$ for $|k| \geq \Delta$ (e.g. WN: $\Delta = 1$)

Then $v(n)$ is uncorrelated with $v(n - \Delta)$, but $s(n)$ is correlated with $s(n - \Delta)$

$\Rightarrow y(n) \cong s(n)$ and $e(n) \cong v(n)$

Application: Adaptive Noise Cancellation



Domain:
Acoustics, Vibration, ...

Operation:

This is (unlike ALE) a 2-channel system

Primary sensor (e.g. MIC) is close to the signal source but also has noise parts

Secondary sensor is close to the noise source (measures a reference to the noise)

The noise components in the 2 sensors are not identical due to (e.g. acoustic) transfer function from noise source to sensors, but are correlated

The adaptive algorithm minimizes $e(n) = d(n) - y(n) = s(n) + x'(n) - y(n)$

Now $E[e^2(n)] = E[s^2(n)] + E[(x'(n) - y(n))^2] + 2E[s(n)x'(n)] - 2E[s(n)y(n)]$
 $= E[s^2(n)] + E[(x'(n) - y(n))^2]$

Minimization: $y(n) = x'(n) \Rightarrow W(z) = P(z) \Rightarrow e(n) = s(n)$ (note that here $e(n) \neq 0$)